JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Calendar.Builder

java.lang.Object
    java.util.Calendar.Builder

Enclosing class:

Calendar

```
public static class Calendar.Builder
extends Object
```

`Calendar.Builder` is used for creating a `Calendar` from various date-time parameters.

There are two ways to set a `Calendar` to a date-time value. One is to set the instant parameter to a millisecond offset from the Epoch. The other is to set individual field parameters, such as `YEAR`, to their desired values. These two ways can't be mixed. Trying to set both the instant and individual fields will cause an `IllegalStateException` to be thrown. However, it is permitted to override previous values of the instant or field parameters.

If no enough field parameters are given for determining date and/or time, calendar specific default values are used when building a `Calendar`. For example, if the `YEAR` value isn't given for the Gregorian calendar, 1970 will be used. If there are any conflicts among field parameters, the resolution rules are applied. Therefore, the order of field setting matters.

In addition to the date-time parameters, the locale, time zone, week definition, and leniency mode parameters can be set.

Examples

The following are sample usages. Sample code assumes that the `Calendar` constants are statically imported.

The following code produces a `Calendar` with date 2012-12-31 (Gregorian) because Monday is the first day of a week with the ISO 8601 compatible week parameters.

```
Calendar cal = new Calendar.Builder().setCalendarType("iso8601")
                     .setWeekDate(2013, 1, MONDAY).build();
```

The following code produces a Japanese `Calendar` with date 1989-01-08 (Gregorian), assuming that the default `ERA` is *Heisei* that started on that day.

```
Calendar cal = new Calendar.Builder().setCalendarType("japanese")
                     .setFields(YEAR, 1, DAY_OF_YEAR, 1).build();
```

Since:

1.8

See Also:

`Calendar.getInstance(TimeZone, Locale)`, `Calendar.fields`

Constructor Summary

Constructors

Constructor	Description
`Builder()`	Constructs a `Calendar.Builder`.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`Calendar`	**`build()`**	Returns a `Calendar` built from the parameters set by the setter methods.
`Calendar.Builder`	**`set`**`(int field, int value)`	Sets the `field` parameter to the given `value`.
`Calendar.Builder`	**`setCalendarType`**`(String type)`	Sets the calendar type parameter to the given `type`.
`Calendar.Builder`	**`setDate`**`(int year, int month, int dayOfMonth)`	Sets the date field parameters to the values given by `year`, `month`, and `dayOfMonth`.
`Calendar.Builder`	**`setFields`**`(int... fieldValuePairs)`	Sets field parameters to their values given by `fieldValuePairs` that are pairs of a field and its value.

Calendar.Builder	**setInstant** (long instant)	Sets the instant parameter to the given instant value that is a millisecond offset from the Epoch.
Calendar.Builder	**setInstant** (Date instant)	Sets the instant parameter to the instant value given by a Date.
Calendar.Builder	**setLenient** (boolean lenient)	Sets the lenient mode parameter to the value given by lenient.
Calendar.Builder	**setLocale** (Locale locale)	Sets the locale parameter to the given locale.
Calendar.Builder	**setTimeOfDay** (int hourOfDay, int minute, int second)	Sets the time of day field parameters to the values given by hourOfDay, minute, and second.
Calendar.Builder	**setTimeOfDay** (int hourOfDay, int minute, int second, int millis)	Sets the time of day field parameters to the values given by hourOfDay, minute, second, and millis.
Calendar.Builder	**setTimeZone** (TimeZone zone)	Sets the time zone parameter to the given zone.
Calendar.Builder	**setWeekDate** (int weekYear, int weekOfYear, int dayOfWeek)	Sets the week-based date parameters to the values with the given date specifiers - week year, week of year, and day of week.
Calendar.Builder	**setWeekDefinition** (int firstDayOfWeek, int minimalDaysInFirstWe	Sets the week definition parameters to the values given by firstDayOfWeek and minimalDaysInFirstWeek

	that are used to determine the first week of a year.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

Builder

`public Builder()`

Constructs a `Calendar.Builder`.

Method Details

setInstant

`public Calendar.Builder setInstant(long instant)`

Sets the instant parameter to the given `instant` value that is a millisecond offset from the Epoch.

Parameters:

`instant` - a millisecond offset from the Epoch

Returns:

this `Calendar.Builder`

Throws:

`IllegalStateException` - if any of the field parameters have already been set

See Also:

`Calendar.setTime(Date)`, `Calendar.setTimeInMillis(long)`, `Calendar.time`

setInstant

```
public Calendar.Builder setInstant(Date instant)
```

Sets the instant parameter to the `instant` value given by a `Date`. This method is equivalent to a call to `setInstant(instant.getTime())`.

Parameters:

`instant` - a `Date` representing a millisecond offset from the Epoch

Returns:

this `Calendar.Builder`

Throws:

`NullPointerException` - if `instant` is `null`

`IllegalStateException` - if any of the field parameters have already been set

See Also:

`Calendar.setTime(Date)`, `Calendar.setTimeInMillis(long)`, `Calendar.time`

set

```
public Calendar.Builder set(int field,
                            int value)
```

Sets the `field` parameter to the given `value`. `field` is an index to the `Calendar.fields`, such as `DAY_OF_MONTH`. Field value validation is not performed in this method. Any out of range values are either normalized in lenient mode or detected as an invalid value in non-lenient mode when building a `Calendar`.

Parameters:

`field` - an index to the `Calendar` fields

`value` - the field value

Returns:

this `Calendar.Builder`

Throws:

`IllegalArgumentException` - if `field` is invalid

`IllegalStateException` - if the instant value has already been set, or if fields have been set too many (approximately `Integer.MAX_VALUE`) times.

See Also:

`Calendar.set(int, int)`

setFields

```
public Calendar.Builder setFields(int... fieldValuePairs)
```

Sets field parameters to their values given by `fieldValuePairs` that are pairs of a field and its value. For example,

```
setFields(Calendar.YEAR, 2013,
          Calendar.MONTH, Calendar.DECEMBER,
          Calendar.DAY_OF_MONTH, 23);
```

is equivalent to the sequence of the following `set` calls:

```
set(Calendar.YEAR, 2013)
.set(Calendar.MONTH, Calendar.DECEMBER)
.set(Calendar.DAY_OF_MONTH, 23);
```

Parameters:

`fieldValuePairs` - field-value pairs

Returns:

this `Calendar.Builder`

Throws:

`NullPointerException` - if `fieldValuePairs` is `null`

`IllegalArgumentException` - if any of fields are invalid, or if `fieldValuePairs.length` is an odd number.

`IllegalStateException` - if the instant value has been set, or if fields have been set too many (approximately `Integer.MAX_VALUE`) times.

setDate

```
public Calendar.Builder setDate(int year,
                                int month,
                                int dayOfMonth)
```

Sets the date field parameters to the values given by `year`, `month`, and `dayOfMonth`. This method is equivalent to a call to:

```
setFields(Calendar.YEAR, year,
          Calendar.MONTH, month,
          Calendar.DAY_OF_MONTH, dayOfMonth);
```

Parameters:

`year` - the `YEAR` value

`month` - the `MONTH` value (the month numbering is *0-based*).

`dayOfMonth` - the `DAY_OF_MONTH` value

Returns:

this `Calendar.Builder`

setTimeOfDay

```
public Calendar.Builder setTimeOfDay(int hourOfDay,
                                     int minute,
                                     int second)
```

Sets the time of day field parameters to the values given by `hourOfDay`, `minute`, and `second`. This method is equivalent to a call to:

```
setTimeOfDay(hourOfDay, minute, second, 0);
```

Parameters:

`hourOfDay` - the `HOUR_OF_DAY` value (24-hour clock)

`minute` - the `MINUTE` value

`second` - the `SECOND` value

Returns:

this `Calendar.Builder`

setTimeOfDay

```
public Calendar.Builder setTimeOfDay(int hourOfDay,
                                     int minute,
                                     int second,
                                     int millis)
```

Sets the time of day field parameters to the values given by `hourOfDay`, `minute`, `second`, and `millis`. This method is equivalent to a call to:

```
setFields(Calendar.HOUR_OF_DAY, hourOfDay,
          Calendar.MINUTE, minute,
          Calendar.SECOND, second,
          Calendar.MILLISECOND, millis);
```

Parameters:

`hourOfDay` - the `HOUR_OF_DAY` value (24-hour clock)

`minute` - the `MINUTE` value

`second` - the `SECOND` value

millis - the MILLISECOND value

Returns:

this Calendar.Builder

setWeekDate

```
public Calendar.Builder setWeekDate(int weekYear,
                                    int weekOfYear,
                                    int dayOfWeek)
```

Sets the week-based date parameters to the values with the given date specifiers - week year, week of year, and day of week.

If the specified calendar doesn't support week dates, the build method will throw an IllegalArgumentException.

Parameters:

weekYear - the week year

weekOfYear - the week number based on weekYear

dayOfWeek - the day of week value: one of the constants for the DAY_OF_WEEK field: SUNDAY, ..., SATURDAY.

Returns:

this Calendar.Builder

See Also:

Calendar.setWeekDate(int, int, int),
Calendar.isWeekDateSupported()

setTimeZone

```
public Calendar.Builder setTimeZone(TimeZone zone)
```

Sets the time zone parameter to the given zone. If no time zone parameter is given to this Calendar.Builder, the default TimeZone will be used in the build method.

Parameters:

zone - the TimeZone

Returns:

this Calendar.Builder

Throws:

NullPointerException - if zone is null

See Also:

Calendar.setTimeZone(TimeZone)

setLenient

```
public Calendar.Builder setLenient(boolean lenient)
```

Sets the lenient mode parameter to the value given by `lenient`. If no lenient parameter is given to this `Calendar.Builder`, lenient mode will be used in the `build` method.

Parameters:

`lenient` - `true` for lenient mode; `false` for non-lenient mode

Returns:

this `Calendar.Builder`

See Also:

`Calendar.setLenient(boolean)`

setCalendarType

```
public Calendar.Builder setCalendarType(String type)
```

Sets the calendar type parameter to the given `type`. The calendar type given by this method has precedence over any explicit or implicit calendar type given by the locale.

In addition to the available calendar types returned by the `Calendar.getAvailableCalendarTypes` method, `"gregorian"` and `"iso8601"` as aliases of `"gregory"` can be used with this method.

Parameters:

`type` - the calendar type

Returns:

this `Calendar.Builder`

Throws:

`NullPointerException` - if `type` is `null`

`IllegalArgumentException` - if `type` is unknown

`IllegalStateException` - if another calendar type has already been set

See Also:

`Calendar.getCalendarType()`,
`Calendar.getAvailableCalendarTypes()`

setLocale

```
public Calendar.Builder setLocale(Locale locale)
```

Sets the locale parameter to the given `locale`. If no locale is given to this `Calendar.Builder`, the default `Locale` for `Locale.Category.FORMAT` will be used.

If no calendar type is explicitly given by a call to the `setCalendarType` method, the `Locale` value is used to determine what type of `Calendar` to be built.

If no week definition parameters are explicitly given by a call to the `setWeekDefinition` method, the `Locale`'s default values are used.

Parameters:

`locale` - the `Locale`

Returns:

this `Calendar.Builder`

Throws:

`NullPointerException` - if `locale` is `null`

See Also:

`Calendar.getInstance(Locale)`

setWeekDefinition

```
public Calendar.Builder setWeekDefinition(int firstDayOfWeek,
                                          int minimalDaysInFirstWeek)
```

Sets the week definition parameters to the values given by `firstDayOfWeek` and `minimalDaysInFirstWeek` that are used to determine the first week of a year. The parameters given by this method have precedence over the default values given by the locale.

Parameters:

`firstDayOfWeek` - the first day of a week; one of `Calendar.SUNDAY` to `Calendar.SATURDAY`

`minimalDaysInFirstWeek` - the minimal number of days in the first week (1..7)

Returns:

this `Calendar.Builder`

Throws:

`IllegalArgumentException` - if `firstDayOfWeek` or `minimalDaysInFirstWeek` is invalid

See Also:

`Calendar.getFirstDayOfWeek()`, `Calendar.getMinimalDaysInFirstWeek()`

build

`public Calendar build()`

Returns a `Calendar` built from the parameters set by the setter methods. The calendar type given by the `setCalendarType` method or the locale is used to determine what `Calendar` to be created. If no explicit calendar type is given, the locale's default calendar is created.

If the calendar type is `"iso8601"`, the Gregorian change date of a `GregorianCalendar` is set to `Date(Long.MIN_VALUE)` to be the *proleptic* Gregorian calendar. Its week definition parameters are also set to be compatible with the ISO 8601 standard. Note that the `getCalendarType` method of a `GregorianCalendar` created with `"iso8601"` returns `"gregory"`.

The default values are used for locale and time zone if these parameters haven't been given explicitly.

If the locale contains the time zone with "tz" Unicode extension, and time zone hasn't been given explicitly, time zone in the locale is used.

Any out of range field values are either normalized in lenient mode or detected as an invalid value in non-lenient mode.

Returns:

a `Calendar` built with parameters of this `Calendar.Builder`

Throws:

`IllegalArgumentException` - if the calendar type is unknown, or if any invalid field values are given in non-lenient mode, or if a week date is given for the calendar type that doesn't support week dates.

See Also:

`Calendar.getInstance(TimeZone, Locale)`,
`Locale.getDefault(Locale.Category)`,
`TimeZone.getDefault()`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.